Document Number: P2835R4.
Date: 2024-05-21.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Authors: Gonzalo Brito Gadeschi, Mark Hoemmen, Carter H. Edwards, Bryce Adelstein Lelbach.
Audience: LEWG.

Expose `std::atomic_ref` 's object address

Changelog

- R4: St. Louis
  - Switched to use `uintptr_t` .
  - (REVERTED) Update the return type from `const void *` to `address_return_t` which copies the cv-qualifiers of T; updated examples, godbolt samples, and returns clause accordingly.
  - Include tokyo poll (https://github.com/cplusplus/papers/issues/1545#issuecomment-2028998320):
    - POLL:_ Forward “P2835R3: Expose std::atomic_ref 's object address” to LWG for C++26 (to be confirmed by an electronic poll).

SF	F	N	A	SA
5	8	3	0	0

Outcome: Consensus in favor

- R3: LEWG mailing list review
  - Per request by one reviewer on the LEWG mailing list, three coauthors of P0019 (Carter H. Edwards, Mark Hoemmen, and Bryce Adelstein Lelbach) joined this proposal to express their approval.
  - Added background section.
  - Added history of why this API was not part of the original paper.
  - Update return type to `const void*`.
  - Rename API to `address()`.
  - Added rationale against `uintptr_t`.
  - Added rationale against `get()` and `data()`.
  - Add new use cases and examples.
  - Removed Discovery Patterns example.
- R2: Preparation for mailing list review
  - Update links to compiler explorer.
  - Update API design rationale.
  - Update `__cpp_lib_atomic_ref` macro.
- R1:
  - Add alternative API designs.
- R0: initial revision (Varna)

Introduction

Applications that need atomic access to an object and want to reason about contention for performance cannot use C++20 `std::atomic_ref`. Some applications may change the object's storage type to `std::atomic`, but `std::atomic_ref`'s raison d'être is that many applications cannot.

This proposal extends `std::atomic_ref` with a member function that returns the object's address. This enables legacy applications that updated their APIs from `volatile int*` to `atomic_ref` to become conforming with the post-C++11 memory model, to recover the optimizations they lost while doing so, while enabling applications still stuck with `volatile*` on their APIs to migrate to use `atomic_ref`.

Before-and-after ("Tony") tables

Before	After

`#include <atomic>`
`#include <cassert>`
 	
`std::atomic<int> data;`
 	
`void fn(std::atomic<int>& ref) {`
`  auto* addr = &ref;`
`  assert( &data == &ref );`
`}`
 	
`int main() {`
`  fn(data);`
`  return 0;`
`}`
	`#include <atomic>`
	`#include <cassert>`
	
	`int data;`
	
	`void fn(std::atomic_ref<int> ref) {`
	`  const void* addr = ref.address();`
	`  assert( &data == ref.address() );`
	`}`
	
	`int main() {`
	`  fn(std::atomic_ref{data});`
	`  return 0;`
	`}`

Motivation

`std::atomic_ref<T>` ensures that all accesses to an existing `T` object are atomic. Unlike with `std::atomic<T>`, for `std::atomic_ref<T>`, the `T` object exists and its lifetime strictly includes all `std::atomic_ref<T>` that refer to it.

Therefore, a `T` object that is used with `std::atomic_ref<T>` could be accessed with both atomic and non-atomic operations during its lifetime (in contrast to `std::atomic<T>`).

However, as long as one or more live `std::atomic_ref`s still reference the `T` object, the object can only be accessed through these `std::atomic_ref`s. That is, non-atomic accesses are not allowed to be concurrent with accesses through `std::atomic_ref`.

> **Note**: this enables implementations of `atomic_ref<T>` to, e.g., copy the value into an `atomic<T>` which is in a different memory location, operate on that, and once the last `atomic_ref` is destroyed, copy the value back (for which implementations must track the address of the original object's location). This proposal does not recommend such an implementation, but it is legal.

The following examples illustrate `atomic_ref` semantics.

This example is well-defined: non-atomic accesses to `data` happen only while there are no live `atomic_ref`s that reference `data`.

```
int data = 13; // Non-Atomic access
{
    assert(data == 13);
    atomic_ref<int> r{data};
    r.store(0); // Atomic access
} // All atomic_refs are destroyed here

// No atomic_refs are live:
assert(data == 0);
data = 42;
```

This example exhibits undefined behavior: a non-atomic access to `data` happens while there is still a live `atomic_ref` that references `data`.

```
int data = 13;           // Non-Atomic access
atomic_ref<int> r{data}; // atomic_ref live
data = 42;               // UB: object accessed during lifetime of atomic_re
```

The implementation of APIs using `std::atomic<T>*` may obtain the object address without breaking API changes.

```
void api_atomic(std::atomic<int>* ptr) {
    // can obtain address of underlying object:
    uintptr_t address = reinterpret_cast<uintptr_t>(ptr);
}
```

The implementation of APIs using `std::atomic_ref<T>` may not:

```
void api_atomic_ref(atomic_ref<int> ref) {
    // ...cannot obtain the address of underlying object
    // if constexpr(sizeof(ref) == sizeof(uintptr_t)) {
    //   uintptr_t address = reinterpret_cast<uintptr_t>(&ref); // UB
    // }
}
```

This proposal extends the `atomic_ref` API with a member function `address()` to obtain the underlying object's address.

```
void api_atomic_ref_this_paper(atomic_ref<int> ref) {
    // With this proposal, one can get the underlying object's address
    uintptr_t address = reinterpret_cast<uintptr_t>(ref.address());
}
```

Intent of `atomic_ref` proposal

The paper that introduced `atomic_ref` in C++ 20 is P0019R8 (https://wg21.link/P0019R8). The authors discussed this use case and decided the application should track `&data` themselves. However, APIs evolve as usage patterns emerge: SG1 reviewed this paper to address this oversight, and forwarded it with *unanimous consent*. Multiple authors of the original `atomic_ref` paper are co-authors of this paper.

Use cases

This proposal enables legacy APIs that are still using `volatile*` to signal concurrency, due to their implementations needing the object's address internally (see Motivation), to finally migrate to the C++11 Memory Model.

Some of the reasons why these APIs need the object address are covered in this section. Others, like "C Foreign Function Interface (FFI)," are not currently covered in this document.

Atomic access to elements of a data structure

Applications that want to perform atomic access to the elements of a data structure need to make the data structure's element type `atomic`,

> ```
> std::array<std::atomic<int>, N> array;
> ```

and use pointers to `atomic` objects to access the elements.

> ```
> int fetch_add_idx(std::atomic<int>* base, size_t i, int value) {
>     return base[i].fetch_add(value);
> }
> ```

When the array is provided externally, e.g., from a third-party C API,
it is typically an array of `T`, not an array of `atomic<T>`.

> ```
> extern int array[N];
> ```

Before `atomic_ref` was introduced in C++20, it was common practice for applications to create APIs that drop the "atomicity" semantics. Such applications would use `volatile` (with nonstandard meaning) to express their intent.

```
int fetch_add_idx(/* not atomic */ int volatile* base, size_t i, int value)
    return std::atomic_ref{base[i]}.fetch_add(value);
}
```

However, it is not possible for applications written in Standard C++ to encode "atomicity" semantics as part of their API without a way to extract the underlying's object address. This proposal provides that way: the `address` member function.

```
int fetch_add_idx(std::atomic_ref<int> base, size_t i, int value) {
    auto p = static_cast<int*>(base.address());
    return std::atomic_ref{*(p+i)}.fetch_add(value);
}
```

This matches the original intended use case of `std::atomic_ref<T>`, namely accessing the elements of an existing array of `T` atomically.

In fact, for the partiular case of contiguous data structures, like the array above, `std::atomic_ref` was specifically designed to be a proxy reference type for `mdspan` accessors. The `atomic_accessor` class in P2689 (https://wg21.link/p2689) has an `access(T* p, size_t i)` member function that returns `std::atomic_ref{*(p+i)}`. The `mdspan` proposal P0009 (https://wg21.link/p0009) used `atomic_ref` and its corresponding accessor as justification for `mdspan` permitting custom accessors. (See e.g., the "Why custom accessors?" section of P0009.)

Contention-aware data-structures

Contention-aware data-structures rely on the object's address to tell different objects apart. The addresses are then used to, e.g., index into global lock tables.

Feedback during LEWG mailing review suggested that examples that were too advanced were unapproachable. R2 of the paper provided a fully working implementation of one such example in compiler-explorer, demonstrating a 1.25x performance improvement from this API. Similar algorithms are part of, e.g., C++ standard library implementations (e.g., here (https://github.com/NVIDIA/cccl/blob/b7d4228ab7268ed928984cd61096079bd671d25d/libcudacxx/include/cuda/std/detail/libcxx/include/__cuda/barrier.h#L231-L250)).

The following example (godbolt (https://gcc.godbolt.org/z/9TebvT4Ge)) illustrate how concurrent algorithms and data structures typically detect and react to contention in practice. First, they obtain the object's address to tell it apart from other objects. They often hash it. Then, they use the (possibly hashed) address to index into global data structures (e.g., lock tables or timer tables) to improve contention. Without this proposal, an API-breaking change would be required in order to apply these optimizations to an API that only accepts an `atomic_ref`.

```
constexpr std::size_t nary = 3;
std::array<std::atomic<std::size_t>, nary> contention;

// Adds one to v (waits until contention is low):
void add_one(std::atomic_ref<int> v) {
    // NEEDS P2835: Get atomic_ref object address:
    auto a = (std::uintptr_t)v.address();

    // Hash address (or cache line address, etc.) by truncating
    // to 32-bit and multiplying by Golden Ratio:
    auto h = ((std::uint32_t)(std::uintptr_t)a) * 2654435761;

    // Check for potential contention:
    auto k = h % nary;
    auto l = contention[k].fetch_add(1);

    // Wait until contention is low (<2 contending threads):
    while (contention[k].load() > 2);

    // Modify object and drop from contention table:
    v.fetch_add(1);
    contention[k].fetch_sub(1);
}
```

Design

The name and return type should prevent accidental misuse that could result from accessing the object through the address while there are still live `atomic_ref` that reference it.

We considered the following options.

- Return type:
  - `uintptr_t`:
    - PRO: Correctly suggest that the value should not be used to access the object.
    - CON: it is an optional type, and therefore APIs using it must be optional.
  - `T const*`:
    - PRO: works.

        - CON: makes it too easy for developers to accidentally access the referenced object while there are still live `atomic_ref` objects, resulting in undefined behavior.
    - `void volatile const*`:
        - PRO: prevents accidental misuse and is not optional.
        - CON: requires casting `const` and `volatile`.
    - `address_return_t`: copies cv-qualifiers of `T`
        - PRO: prevents accidental misuse and is not optional.
        - CON: quite complex, for no good reason:

```
using address_return_t
  = conditional_t<is_const_v<T> && is_volatile_v<T>, void const volatile*,
    conditional_t<is_const_v<T>, void const*,
    conditional_t<is_volatile_v<T>, void volatile*,
    void*>>>;
```

- Name:
    - `data`: Throughout the Standard Library, types with a `data` member function always have a `size` member function. The two functions together indicate that the type represents a contiguous range. A `data` member function would thus incorrectly suggest that `atomic_ref` also represents a contiguous range. Also, `mdspan` deliberately rejected the name `data` in favor of `data_handle`, because `mdspan` might not actually view a contiguous range and because `data_handle_type` might not necessarily be `element_type*`.
    - `get`: All Standard Library types with a `get` member function return a pointer.
    - `ref`: A name with "ref" or "reference" in it would incorrectly suggest that the return value may be used to access the object, just like `std::reference_wrapper`.
    - `address`: Indicates that the intent of the API is returning the object's address only. Does not suggest that the address may be used to access the object. (Returning `uintptr_t` would suggest this even more strongly, but this is not possible, for the reasons listed above.)
    - `unsafe_address`: This would be more explicit. However, there is nothing "unsafe" about getting the address; only accessing the object through the address would be unsafe.

This design proposes using `uintptr_t` as the return type, and `address` as the name. This makes this API optional, conditional on `uintptr_t` availability.

If C++ were to require `uintptr_t` (e.g. see P3248 ()), i.e., `uintptr_t` becomes non-optional, then this API should become non-optional.

Impact on implementations

This proposal does not impact any implementation we are aware of. We surveyed libc++, libstdc++, Microsoft STL, and libcu++.

Wording

Add the following to [atomics.ref.generic.general] (http://eel.is/c++draft/atomics.ref.generic.general).

```
namespace std {
  template<class T> struct atomic_ref {
    private:
        T* ptr;  // exposition only
    public:
    // ...
    uintptr_t address() const noexcept; // optional
    // ...
  };
}
```

Add the following to [atomic.ref.ops] (http://eel.is/c++draft/atomics.ref.ops):

```
uintptr_t address() const noexcept;
```

Returns: `(uintptr_t)ptr .`

Update `__cpp_lib_atomic_ref` version macro in `<version>` synopsis [version.syn] (https://eel.is/c++draft/version.syn#2) to the C++ version this feature is introduced in:

```
#define __cpp_lib_atomic_ref ~~201806~~______L // freestanding, also in <atomic>
```